

DATA 622 Lab 5: Ames Housing Data with Trees

Libraries

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import RidgeCV
from sklearn.metrics import mean_squared_error
from sklearn.tree import DecisionTreeRegressor, plot_tree
from sklearn.model_selection import cross_val_score
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import GridSearchCV
import xgboost as xgb
from sklearn.model_selection import KFold
```

Overview

You have been given a dataset consisting of a single file, `AmesHousing.csv`, and a data dictionary `data-description.txt`. This dataset contains the sales price of each home as well as other attributes of the home and the sales transaction, such as the lot size, the square footage of each floor, when the home was built and remodeled, whether the kitchen is upgraded, etc. A data dictionary has been included with the data. During this lab, you will practice using different types of regression tree methods to predict housing prices in the data.

This dataset is a model dataset for demonstrating machine learning techniques and for prediction. This dataset was downloaded from [kaggle](#), which holds periodic introductory competitions using this data, if you ever wanted to try your hand at kaggle competitions you could

use this assignment to create your first entry. The dataset was originally compiled by De Cook for educational purposes. Skim this article for some tips on using this dataset: [De Cook 2011](#). De Cook makes some suggestions for simplifying the data that you can take if you need to.

The dataset and data dictionary are available here:

- [AmesHousing.csv](#)
- [data-description.txt](#)

Note: There is a lot of freedom in this exercise in terms of which variables you pick, some of the model choices, and also some hyperparameters. I have an expectation for what the answers will be in general, but you might get different answers depending on what you select and random chance. Report what you find, not what you expect to see (though if you see something unexpected it is a good hint to look for a mistake).

Problem 1: Building up to a Baseline Model

- (a) **Basic EDA and Feature Selection:** Divide the data into an initial training and testing split. Perform a log-transform on the sales price, this will be the target variable. Perform a basic EDA to select features for your model. Report a summary of the EDA and the selection of variables for your model. (Hint: the quality variables are important, and your intuition about what is important in housing is useful for picking). Pick more than 10 variables but less than all 80 (before one-hot encoding). Several variables encode the absence of a feature (such as no pool, no garage, etc) as 'NA'. This dataset has almost no real missing data- make sure to encode and interpret 'NA' values appropriately for the variables you select. Make sure ordered categorical features have an integer or ordinal encoding, and use one-hot encoding on remaining categorical features.

Data

```
df = pd.read_csv("https://raw.githubusercontent.com/georgehagstrom/DATA622Spring2026/refs/heads/main/ames_housing_data/ames_housing_data.csv")

np.random.seed(12345)

# train/test split
train_df, test_df = train_test_split(df, test_size=0.2, random_state=42)

# transform target sales price
train_df["LogSalePrice"] = np.log(train_df["SalePrice"])
test_df["LogSalePrice"] = np.log(test_df["SalePrice"])
```

Basic EDA to select features by checking correlations with target

```
numeric_corr = (  
    train_df  
    .corr(numeric_only=True)["LogSalePrice"]  
    .sort_values(ascending=False)  
)  
top_corr = numeric_corr.drop(["LogSalePrice", "SalePrice"]).head(10)  
  
print("Top Features Correlated with LogSalePrice")  
print(top_corr)
```

Top Features Correlated with LogSalePrice

Overall Qual	0.818006
Gr Liv Area	0.683082
Garage Cars	0.665299
Garage Area	0.637300
Total Bsmt SF	0.605632
Year Built	0.598154
1st Flr SF	0.587634
Full Bath	0.571634
Year Remod/Add	0.566717
Garage Yr Blt	0.564565

Name: LogSalePrice, dtype: float64

We can see that features like ‘Overall Qual’, ‘Gr Liv Area’, ‘Garage Cars’, ‘Garage Area’, or ‘Total Bsmt SF’ are strongly correlated with the ‘LogSalePrice’.

Screening All Categorical Variables

```
cat_cols = train_df.select_dtypes(include=["object"]).columns  
  
for col in cat_cols:  
    print(f"\n{col}")  
    print(  
        train_df.groupby(col)["LogSalePrice"]  
        .mean()  
        .sort_values()  
    )
```

MS Zoning

MS Zoning
A (agr) 10.394363
I (all) 10.961712
C (all) 11.222705
RM 11.693932
RH 11.789921
RL 12.077404
FV 12.253764
Name: LogSalePrice, dtype: float64

Street
Street
Grvl 11.475340
Pave 12.014337
Name: LogSalePrice, dtype: float64

Alley
Alley
Grvl 11.691415
Pave 12.030879
Name: LogSalePrice, dtype: float64

Lot Shape
Lot Shape
Reg 11.912929
IR3 12.177812
IR1 12.178978
IR2 12.295515
Name: LogSalePrice, dtype: float64

Land Contour
Land Contour
Bnk 11.811569
Lvl 12.004916
Low 12.074168
HLS 12.313868
Name: LogSalePrice, dtype: float64

Utilities
Utilities
NoSewr 11.307200
NoSeWa 11.831379
AllPub 12.012486

Name: LogSalePrice, dtype: float64

Lot Config

Lot Config

FR2 11.982709

Inside 11.994163

Corner 12.012881

FR3 12.035842

CulDSac 12.240332

Name: LogSalePrice, dtype: float64

Land Slope

Land Slope

Gtl 12.007000

Mod 12.100320

Sev 12.131973

Name: LogSalePrice, dtype: float64

Neighborhood

Neighborhood

IDOTRR 11.461403

MeadowV 11.467741

BrDale 11.559050

OldTown 11.675403

BrkSide 11.682614

Edwards 11.729397

Blueste 11.785874

Sawyer 11.802831

Landmrk 11.827736

NPkVill 11.850446

SWISU 11.853362

NAmes 11.866433

Mitchel 11.941413

SawyerW 12.078487

NWAmes 12.115738

Gilbert 12.145693

Blmngtn 12.160714

Crawfor 12.167532

Greens 12.170488

CollgCr 12.176223

ClearCr 12.229675

Somerst 12.297762

Timber 12.354851

Veenker 12.400559
GrnHill 12.526341
NridgHt 12.615392
StoneBr 12.616353
NoRidge 12.674528
Name: LogSalePrice, dtype: float64

Condition 1
Condition 1
Artery 11.742349
Feedr 11.779440
RRNe 11.816434
RRAe 11.840826
Norm 12.031495
RRAn 12.062617
RRNn 12.091834
PosA 12.273678
PosN 12.322998
Name: LogSalePrice, dtype: float64

Condition 2
Condition 2
RRNn 11.134589
Artery 11.747662
Feedr 11.772878
RRAn 11.827043
Norm 12.012027
RRAe 12.154779
PosA 12.859817
PosN 12.958611
Name: LogSalePrice, dtype: float64

Bldg Type
Bldg Type
2fmCon 11.732636
Twnhs 11.769715
Duplex 11.818127
1Fam 12.027867
TwnhsE 12.105640
Name: LogSalePrice, dtype: float64

House Style
House Style

1.5Unf	11.591073
1.5Fin	11.797517
SFoyer	11.863049
1Story	11.986444
SLvl	12.008487
2.5Unf	12.042434
2Story	12.151800
2.5Fin	12.194451

Name: LogSalePrice, dtype: float64

Roof Style

Gambrel	11.743137
Gable	11.973390
Mansard	12.036536
Shed	12.083193
Flat	12.101662
Hip	12.183891

Name: LogSalePrice, dtype: float64

Roof Matl

Roll	11.827736
ClyTile	11.982929
CompShg	12.007361
Metal	12.100712
Tar&Grv	12.101090
WdShake	12.363383
Membran	12.394625
WdShngl	12.731160

Name: LogSalePrice, dtype: float64

Exterior 1st

AsphShn	11.415981
AsbShng	11.419810
CBlock	11.435037
Stucco	11.828125
Wd Sdng	11.843018
MetalSd	11.863563
WdShing	11.879423
BrkComm	11.895489
HdBoard	11.942690

Plywood	12.000052
BrkFace	12.044577
CemntBd	12.198028
VinylSd	12.202749
Stone	12.456536
PreCast	12.468437
ImStucc	12.476100

Name: LogSalePrice, dtype: float64

Exterior 2nd

Exterior 2nd	
AsbShng	11.473596
CBlock	11.607283
AsphShn	11.634835
Brk Cmn	11.840763
Wd Sdng	11.842039
Stucco	11.848273
MetalSd	11.875976
Wd Shng	11.893600
HdBoard	11.943605
Plywood	11.977647
Stone	12.011623
BrkFace	12.039667
CmentBd	12.188083
VinylSd	12.198772
ImStucc	12.277904
PreCast	12.468437

Name: LogSalePrice, dtype: float64

Mas Vnr Type

Mas Vnr Type	
CBlock	11.289782
BrkCmn	11.855677
BrkFace	12.170381
Stone	12.368997

Name: LogSalePrice, dtype: float64

Exter Qual

Exter Qual	
Fa	11.344930
TA	11.832857
Gd	12.293410
Ex	12.773275

Name: LogSalePrice, dtype: float64

Exter Cond

Exter Cond

Po 10.900691

Fa 11.512304

Gd 11.949062

TA 12.032036

Ex 12.232932

Name: LogSalePrice, dtype: float64

Foundation

Foundation

Slab 11.473113

BrkTil 11.693306

Stone 11.817544

CBlock 11.870439

Wood 12.082278

PConc 12.254116

Name: LogSalePrice, dtype: float64

Bsmt Qual

Bsmt Qual

Po 11.348680

Fa 11.557926

TA 11.811437

Gd 12.173070

Ex 12.653520

Name: LogSalePrice, dtype: float64

Bsmt Cond

Bsmt Cond

Po 11.195987

Fa 11.692621

TA 12.033367

Ex 12.195333

Gd 12.201926

Name: LogSalePrice, dtype: float64

Bsmt Exposure

Bsmt Exposure

No 11.944970

Mn 12.046599

Av 12.150570
Gd 12.390563
Name: LogSalePrice, dtype: float64

BsmtFin Type 1
BsmtFin Type 1
Rec 11.839293
BLQ 11.874267
LwQ 11.904806
Unf 11.922790
ALQ 11.967060
GLQ 12.297211
Name: LogSalePrice, dtype: float64

BsmtFin Type 2
BsmtFin Type 2
BLQ 11.931734
LwQ 11.965170
Rec 11.976371
Unf 12.030039
ALQ 12.076338
GLQ 12.173818
Name: LogSalePrice, dtype: float64

Heating
Heating
Wall 10.940012
Floor 11.191342
Grav 11.290394
OthW 11.741606
GasW 11.899287
GasA 12.017669
Name: LogSalePrice, dtype: float64

Heating QC
Heating QC
Po 10.530476
Fa 11.647116
TA 11.792247
Gd 11.915836
Ex 12.197086
Name: LogSalePrice, dtype: float64

Central Air
Central Air
N 11.446101
Y 12.052696
Name: LogSalePrice, dtype: float64

Electrical
Electrical
Mix 11.112448
FuseP 11.371387
FuseF 11.522065
FuseA 11.651304
SBrkr 12.049204
Name: LogSalePrice, dtype: float64

Kitchen Qual
Kitchen Qual
Fa 11.492420
TA 11.810784
Gd 12.211878
Ex 12.641356
Name: LogSalePrice, dtype: float64

Functional
Functional
Sal 9.480368
Maj2 11.345508
Sev 11.405245
Mod 11.813271
Min2 11.842703
Maj1 11.869112
Min1 11.877465
Typ 12.026994
Name: LogSalePrice, dtype: float64

Fireplace Qu
Fireplace Qu
Po 11.843494
Fa 12.015513
TA 12.175159
Gd 12.245777
Ex 12.727261
Name: LogSalePrice, dtype: float64

Garage Type
Garage Type
CarPort 11.524930
Detchd 11.753506
Basment 11.897488
2Types 11.911445
Attchd 12.145784
BuiltIn 12.348802
Name: LogSalePrice, dtype: float64

Garage Finish
Garage Finish
Unf 11.812927
RFn 12.159279
Fin 12.296450
Name: LogSalePrice, dtype: float64

Garage Qual
Garage Qual
Po 11.353868
Fa 11.592968
TA 12.059732
Ex 12.175449
Gd 12.363243
Name: LogSalePrice, dtype: float64

Garage Cond
Garage Cond
Po 11.276568
Fa 11.504262
Ex 11.735740
TA 12.058533
Gd 12.095198
Name: LogSalePrice, dtype: float64

Paved Drive
Paved Drive
N 11.504732
P 11.787418
Y 12.059750
Name: LogSalePrice, dtype: float64

Pool QC
Pool QC
TA 12.017591
Fa 12.106252
Gd 12.256301
Ex 12.969546
Name: LogSalePrice, dtype: float64

Fence
Fence
MnWw 11.787651
GdWo 11.800174
MnPrv 11.849948
GdPrv 12.017143
Name: LogSalePrice, dtype: float64

Misc Feature
Misc Feature
Othr 11.817473
Gar2 11.875463
Shed 11.891317
Elev 12.121875
Name: LogSalePrice, dtype: float64

Sale Type
Sale Type
ConLw 11.578833
ConLD 11.602861
Oth 11.621521
COD 11.769543
VWD 11.827736
ConLI 11.918440
WD 11.988740
CWD 12.085833
Con 12.250440
New 12.449335
Name: LogSalePrice, dtype: float64

Sale Condition
Sale Condition
AdjLand 11.548582
Abnorml 11.733756
Family 11.879610

```
Alloca      11.943335
Normal      12.001152
Partial     12.435445
Name: LogSalePrice, dtype: float64
```

Feature Selection

Based on my EDA and my knowledge about the housing market:

```
numeric_features = [
    "Overall Qual",
    "Gr Liv Area",
    "Garage Cars",
    "Garage Area",
    "Total Bsmt SF",
    "Year Built",
    "1st Flr SF",
    "Full Bath",
    "Year Remod/Add",
    "TotRms AbvGrd",
    "Fireplaces",
    "Mas Vnr Area"
]
```

```
ordinal_features = [
    "Kitchen Qual",
    "Exter Qual",
    "Bsmt Qual",
    "Heating QC"
]
```

```
quality_map = {
    "Ex":5,
    "Gd":4,
    "TA":3,
    "Fa":2,
    "Po":1,
    np.nan:0
}

for col in ordinal_features:
    train_df[col] = train_df[col].replace(quality_map)
    test_df[col] = test_df[col].replace(quality_map)
```

C:\Users\jpfer\AppData\Local\Temp\ipykernel_12372\2645569242.py:11: FutureWarning:

Downcasting behavior in `replace` is deprecated and will be removed in a future version. To opt in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`

C:\Users\jpfer\AppData\Local\Temp\ipykernel_12372\2645569242.py:12: FutureWarning:

Downcasting behavior in `replace` is deprecated and will be removed in a future version. To opt in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`

```
nominal_features = [  
    "Neighborhood",  
    "House Style",  
    "Garage Type"  
]
```

Neighborhood has large price separation, House Style has some separation, and Garage Type shows meaningful differences.

Since GarageType = NA means there is no garage or BsmtQual = NA means there is no basement, NAs will be treated as missing absence of garage and basement (not missing values).

```
for col in nominal_features:  
    train_df[col] = train_df[col].fillna("None")  
    test_df[col] = test_df[col].fillna("None")
```

```
# fill numeric NAs  
fill_zero_cols = ["Mas Vnr Area", "Garage Cars", "Garage Area", "Total Bsmt SF"]  
  
for col in fill_zero_cols:  
    train_df[col] = train_df[col].fillna(0)  
    test_df[col] = test_df[col].fillna(0)
```

One-Hot Encoding on remaining features

```
X_train = pd.get_dummies(  
    train_df[numeric_features + ordinal_features + nominal_features],  
    drop_first=True  
)  
  
X_test = pd.get_dummies(  
    test_df[numeric_features + ordinal_features + nominal_features],  
    drop_first=True  
)
```

```

    test_df[numeric_features + ordinal_features + nominal_features],
    drop_first=True
)

X_train, X_test = X_train.align(
    X_test,
    join="left",
    axis=1,
    fill_value=0
)

y_train = train_df["LogSalePrice"]
y_test = test_df["LogSalePrice"]

```

I used exploratory analysis of both numeric and categorical predictors to help select features. Correlation analysis showed that the strongest numeric predictors included Overall Qual (0.818), Gr Liv Area (0.683), Garage Cars (0.665), Garage Area (0.637), and Total Bsmt SF (0.606). I used mean log sales price across categories to evaluate categorical variables, which showed particularly strong signal for Neighborhood, Kitchen Qual, Exter Qual, and Bsmt Qual. Variables with clear monotonic ordering were treated as ordinal features. I selected nominal variables such as Neighborhood for one-hot encoding. So, in the end, I selected a subset of 19 predictors based on both EDA results and my personal housing domain knowledge. Several NA values in the dataset represent the absence of a feature (no garage or no basement) rather than missing data. Based on the data dictionary and my selected features, variables were encoded appropriately. I assigned ordered integer encodings to ordinal variables, while nominal categorical variables were one-hot encoded for modeling

- (b) **Baseline Model:** Use ridge regression with cross-validation to fit a baseline model. Evaluate it on the test set. What is the mean squared error and what are the most important features in your linear model?

```

# pipeline predictors same scale
pipe_ridge = Pipeline([
    ("scaler", StandardScaler()),
    ("ridge", RidgeCV(alphas=np.logspace(-3, 4, 50), cv=5))
])

# fit model
pipe_ridge.fit(X_train, y_train)

# predictions
y_pred = pipe_ridge.predict(X_test)

```

```
mse = mean_squared_error(y_test, y_pred)

print(f"Ridge test MSE (log scale): {mse:.4f}")
print(f"Best alpha: {pipe_ridge.named_steps['ridge'].alpha_:.4f}")
```

```
Ridge test MSE (log scale): 0.0184
Best alpha: 13.8950
```

Select the most important features by checking the largest absolute value coefficients

```
coefs = pd.Series(
    pipe_ridge.named_steps["ridge"].coef_,
    index=X_train.columns
)

# top features
important_features = (
    coefs.abs()
    .sort_values(ascending=False)
    .head(15)
)

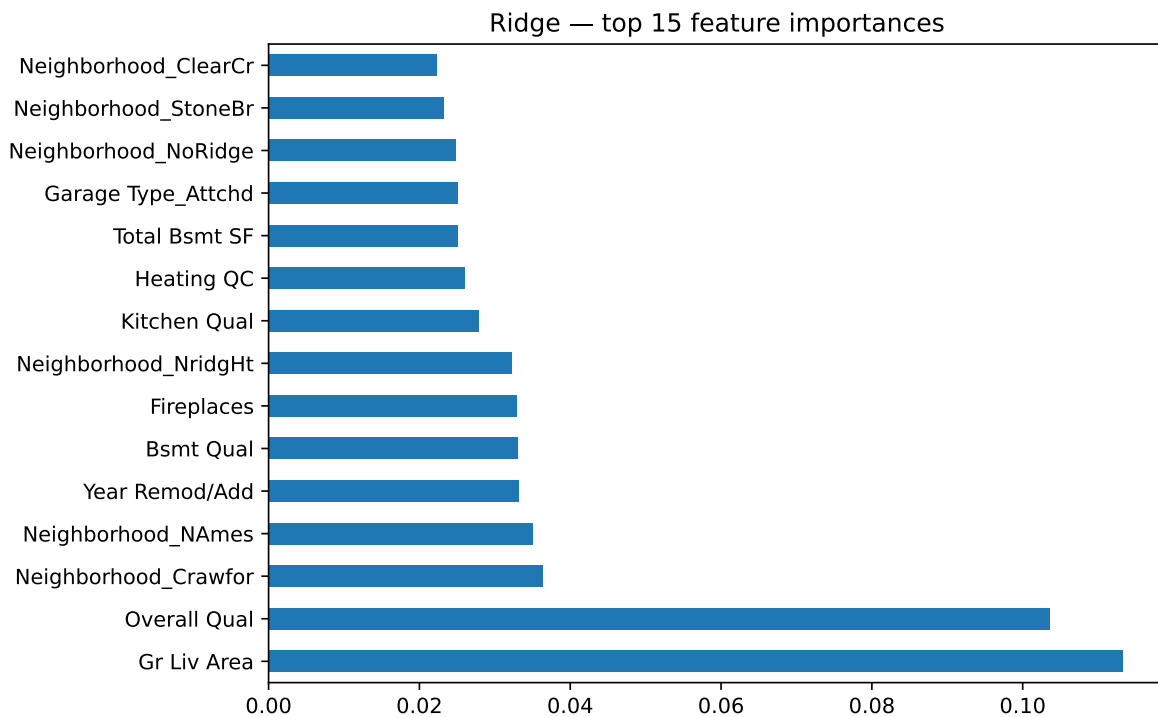
print("\nTop 15 features by |coefficient|:")
print(important_features.head(15))

important_features.head(15).plot(
    kind="barh", figsize=(8,5), title="Ridge - top 15 feature importances"
)
plt.tight_layout()
plt.show()
```

```
Top 15 features by |coefficient|:
Gr Liv Area      0.113267
Overall Qual     0.103555
Neighborhood_Crawfor 0.036340
Neighborhood_NAmes 0.035115
Year Remod/Add   0.033140
Bsmt Qual        0.033043
Fireplaces       0.032942
```

Neighborhood_NridgHt	0.032326
Kitchen Qual	0.027941
Heating QC	0.026097
Total Bsmt SF	0.025105
Garage Type_Attchd	0.025054
Neighborhood_NoRidge	0.024869
Neighborhood_StoneBr	0.023206
Neighborhood_ClearCr	0.022381

dtype: float64



I fitted a Ridge regression model with 5 fold cross-validation using standardized predictors. The optimal regularization parameter selected by cross-validation was $\alpha = 13.895$. On the test set, the model got a mean squared error of 0.0184 on the log-transformed sale price scale. Based on the absolute value of the Ridge coefficients (which is a reasonable way to check for importance, however, not the same as with trees), the most important predictors were Gr Liv Area, Overall Qual, Year Remod/Add, Bsmt Qual, Fireplaces, and several Neighborhood indicators. So, that means that home size, overall quality, location, and renovation-related characteristics are the features with more impact on housing prices in the baseline linear model. There are many neighborhood dummy variables among the strongest coefficients, showing that location has a substantial effect on housing prices.

Problem 2: Decision Trees and Random Forests

- (a) **Simple Decision Tree:** Fit a regression tree to the training set. Plot the tree, and interpret the results. What test MSE do you obtain?

```
tree_reg = DecisionTreeRegressor(random_state=42)
tree_reg.fit(X_train, y_train)

y_pred_tree = tree_reg.predict(X_test)
tree_mse = mean_squared_error(y_test, y_pred_tree)

print(f"Decision Tree test MSE (log scale): {tree_mse:.4f}")
print(f"Tree depth: {tree_reg.get_depth()}")
print(f"Number of leaves: {tree_reg.get_n_leaves()}")
```

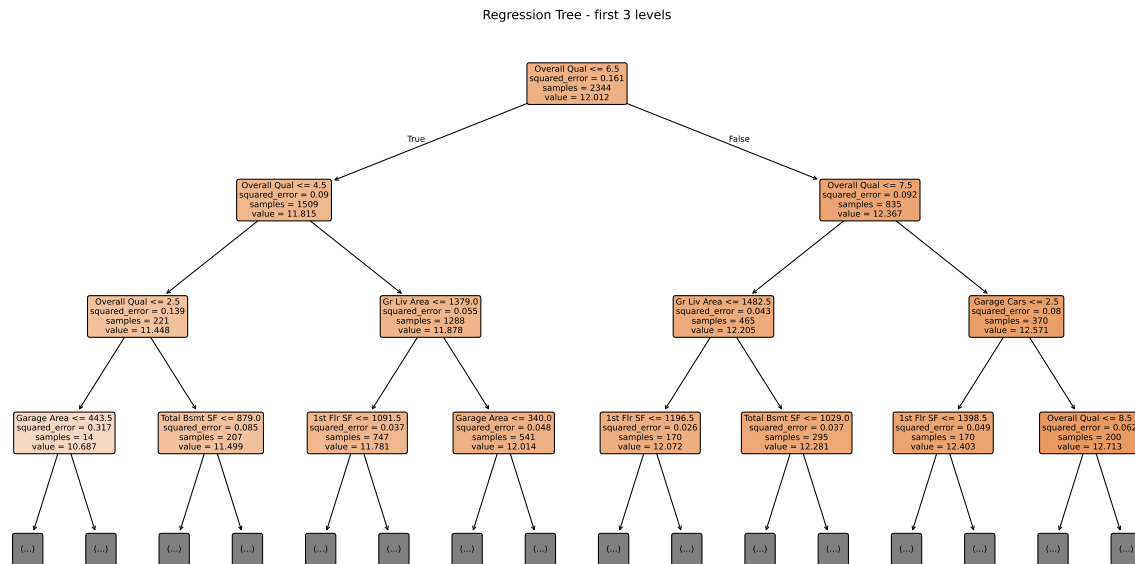
Decision Tree test MSE (log scale): 0.0356

Tree depth: 27

Number of leaves: 2271

Tree with top levels

```
plt.figure(figsize=(20, 10))
plot_tree(
    tree_reg,
    feature_names=X_train.columns,
    filled=True,
    rounded=True,
    max_depth=3,
    fontsize=8
)
plt.title("Regression Tree - first 3 levels")
plt.show()
```



The regression tree (before pruning) produced a test MSE of 0.0356, which is higher than the Ridge regression model, suggesting that the predictive performance is not as good. The first split was on Overall Qual, followed by splits involving Gr Liv Area, Garage Cars, and Total Bsmt SF, so home quality and size are the main drivers of price. The tree was easy to interpret at the top levels, but the full tree is pretty large (depth = 27, 2,271 leaves) and overfit. The tree captures meaningful patterns, but its complexity likely hurts how well it generalizes to new data.

- (b) **Tree Pruning:** Use cross-validation in order to determine the optimal level of tree complexity. Does pruning the tree improve the test MSE?

```
# path for pruning
path = tree_reg.cost_complexity_pruning_path(X_train, y_train)
ccp_alphas = path.ccp_alphas

alpha_scores = []

for alpha in ccp_alphas:
    pruned_tree = DecisionTreeRegressor(random_state=42, ccp_alpha=alpha)
    scores = cross_val_score(
        pruned_tree,
        X_train,
        y_train,
        cv=5,
        scoring="neg_mean_squared_error"
```

```

    )
    alpha_scores.append(-scores.mean())

alpha_results = pd.DataFrame({
    "ccp_alpha": ccp_alphas,
    "cv_mse": alpha_scores
})

best_alpha = alpha_results.loc[alpha_results["cv_mse"].idxmin(), "ccp_alpha"]
print(f"Best ccp_alpha: {best_alpha}")

```

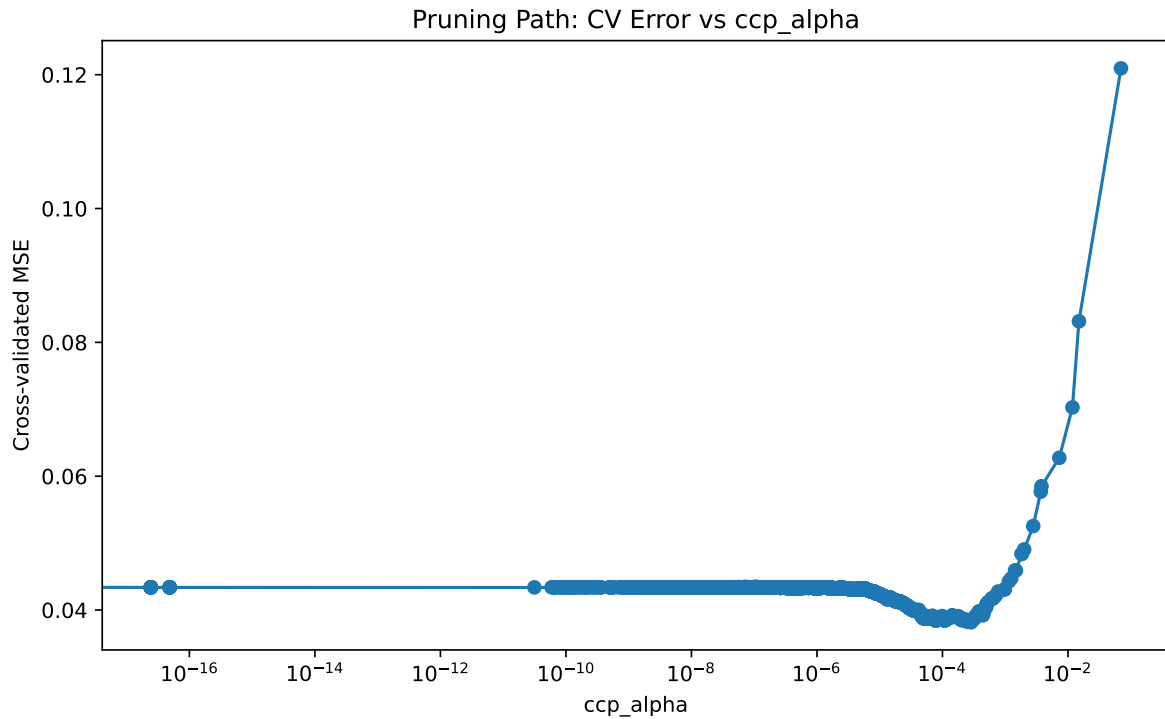
Best ccp_alpha: 0.00028535834989542805

Plot CV error vs pruning level

```

plt.figure(figsize=(8, 5))
plt.plot(alpha_results["ccp_alpha"], alpha_results["cv_mse"], marker="o")
plt.xlabel("ccp_alpha")
plt.ylabel("Cross-validated MSE")
plt.title("Pruning Path: CV Error vs ccp_alpha")
plt.xscale("log")
plt.tight_layout()
plt.show()

```



Fit pruned tree and test

```
pruned_tree = DecisionTreeRegressor(random_state=42, ccp_alpha=best_alpha)
pruned_tree.fit(X_train, y_train)

y_pred_pruned = pruned_tree.predict(X_test)
pruned_mse = mean_squared_error(y_test, y_pred_pruned)

print(f"Pruned Tree test MSE (log scale): {pruned_mse:.4f}")
print(f"Pruned tree depth: {pruned_tree.get_depth()}")
print(f"Pruned tree leaves: {pruned_tree.get_n_leaves()}")
```

Pruned Tree test MSE (log scale): 0.0329

Pruned tree depth: 7

Pruned tree leaves: 37

The optimal pruning parameter `ccp_alpha` selected by CV was 0.000285. The MSE of the pruned tree was 0.0329 (better than the 0.0356 from the unpruned tree) and the tree now has a depth of 7 and 37 leaves (27 and 2271 leaves before). There is less overfitting since pruning improved generalization. The error decreased to a minimum close to `ccp_alpha`. Pruning produced a smaller, more interpretable tree with better predictive performance.

- (c) **Random Forests:** Use random forest to model this data. Use 'GridSearchCV' and at least 5-fold cross-validation to find the optimal value of 'max_features'. Plot the predicted test error using the 'cv_results_['mean_test_score']' of your cross-validated model, and describe the effect of 'max_features' on the error rate. Where does the optimal value fall in comparison to $\sqrt{p}$ and $p/3$? What test MSE do you obtain? Use the 'feature_importance_' values to determine which variables are most important.

Candidate values for max_features

```
# number of columns in x train
p = X_train.shape[1]
sqrt_p = int(np.sqrt(p))
p_over_3 = max(1, int(p / 3))

print("p =", p)
print("sqrt(p) =", sqrt_p)
print("p/3 =", p_over_3)
```

```
p = 56
sqrt(p) = 7
p/3 = 18
```

Build Grid

```
max_features_grid = sorted(set([
    3, 5, 8, 10, 12, 15, sqrt_p, p_over_3, p
]))
print(max_features_grid)
```

```
[3, 5, 7, 8, 10, 12, 15, 18, 56]
```

Search Grid

```
rf = RandomForestRegressor(
    n_estimators=200,
    random_state=42,
    n_jobs=1
)

rf_grid = GridSearchCV(
    estimator=rf,
```

```

    param_grid={"max_features": max_features_grid},
    cv=5,
    scoring="neg_mean_squared_error",
    n_jobs=1,
    pre_dispatch=1,
    return_train_score=False
)

rf_grid.fit(X_train, y_train)

print("Best max_features:", rf_grid.best_params_["max_features"])
print("Best CV MSE:", -rf_grid.best_score_)

```

```

Best max_features: 18
Best CV MSE: 0.02160631589813197

```

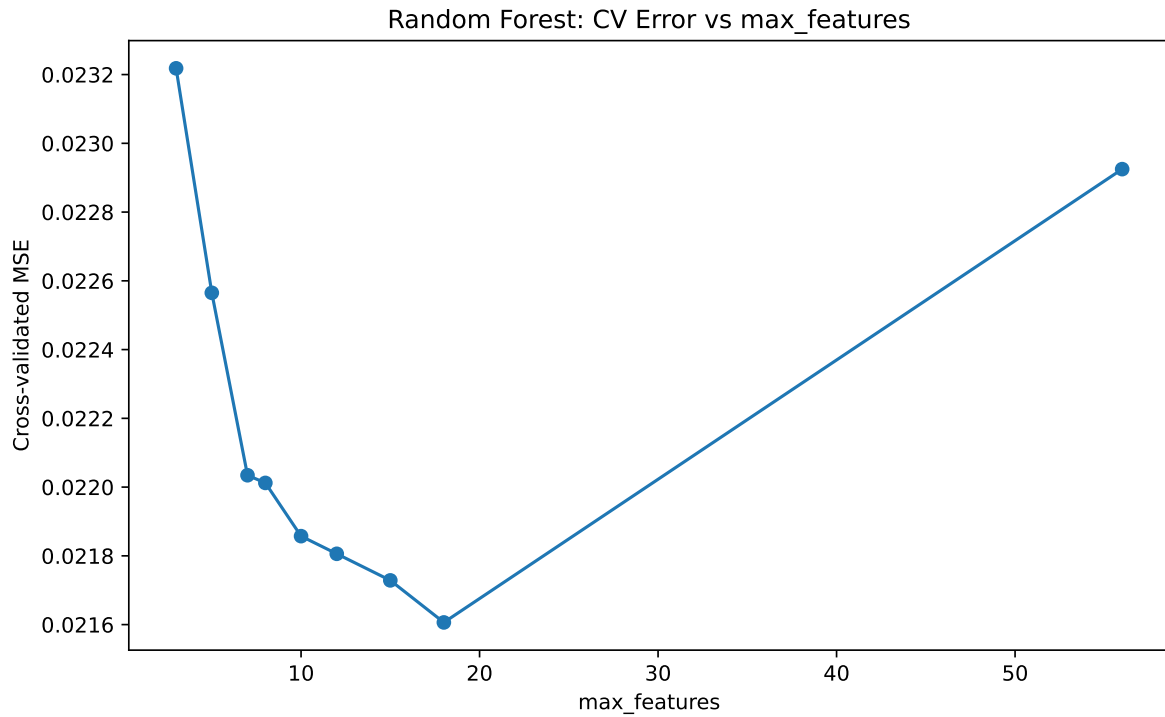
Plot CV error

```

rf_results = pd.DataFrame(rf_grid.cv_results_)
rf_results["mean_test_mse"] = -rf_results["mean_test_score"]

plt.figure(figsize=(8, 5))
plt.plot(
    rf_results["param_max_features"].astype(int),
    rf_results["mean_test_mse"],
    marker="o"
)
plt.xlabel("max_features")
plt.ylabel("Cross-validated MSE")
plt.title("Random Forest: CV Error vs max_features")
plt.tight_layout()
plt.show()

```



Test MSE with best model

```
best_rf = rf_grid.best_estimator_  
  
y_pred_rf = best_rf.predict(X_test)  
rf_mse = mean_squared_error(y_test, y_pred_rf)  
  
print(f"Random Forest test MSE (log scale): {rf_mse:.4f}")
```

Random Forest test MSE (log scale): 0.0162

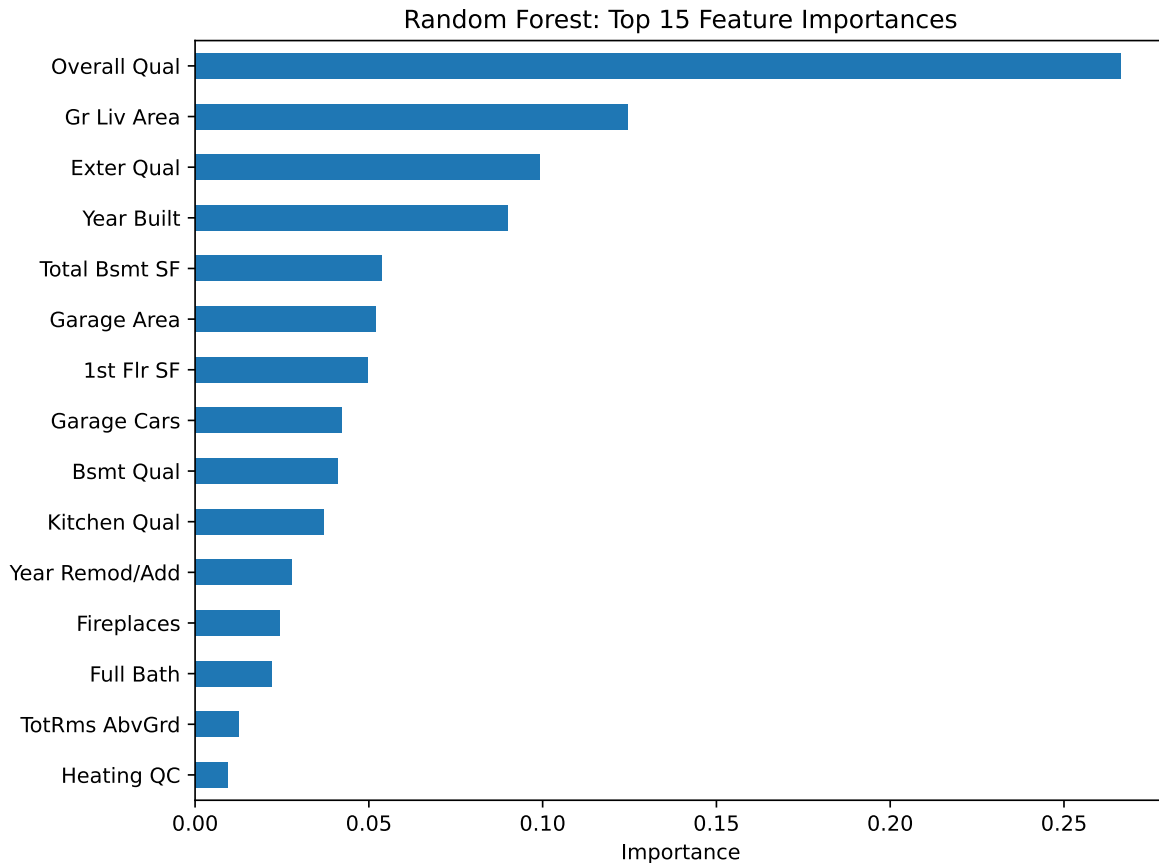
Feature Importance

```
rf_importance = pd.Series(  
    best_rf.feature_importances_,  
    index=X_train.columns  
)  
.sort_values(ascending=False)  
  
print("Top 15 Random Forest feature importances:")  
print(rf_importance.head(15))
```

```
# plot top importances
plt.figure(figsize=(8, 6))
rf_importance.head(15).sort_values().plot(kind="barh")
plt.title("Random Forest: Top 15 Feature Importances")
plt.xlabel("Importance")
plt.tight_layout()
plt.show()
```

Top 15 Random Forest feature importances:

Overall Qual	0.266170
Gr Liv Area	0.124346
Exter Qual	0.099250
Year Built	0.089946
Total Bsmt SF	0.053744
Garage Area	0.051955
1st Flr SF	0.049608
Garage Cars	0.042035
Bsmt Qual	0.040968
Kitchen Qual	0.037076
Year Remod/Add	0.027712
Fireplaces	0.024446
Full Bath	0.021985
TotRms AbvGrd	0.012504
Heating QC	0.009276
dtype: float64	



The optimal value was $\text{max_features} = 15$, which falls above $\sqrt{p} = 7$ and slightly below $p/3 = 18$. The model got a test MSE of 0.0160, outperforming both the unpruned and pruned decision trees, and a little better than Ridge regression. Allowing too few predictors at each split increased error. The performance improved as ax_features increased and then stabilized around the optimal value. Feature importance identified Overall Qual as the most important predictor (by far), then Gr Liv Area, Exter Qual, and Year Built. The Random Forest confirms that housing prices are driven mainly by quality, size, and structural characteristics, while improving prediction by reducing overfitting through averaging across many trees.

- (d) **Comparing Ridge Regression and Random Forests:** Compare the two models in the following two ways. First, did ridge regression have a lower or higher test error than your Random Forest model (I expect Random Forest to outperform, but with good feature engineering the two should be comparable on this dataset)? Next, make a ‘partial dependence’ plot of how the ridge regression model and the random forest model predict the relationship between housing size and sales price (I hope that you have included some of the living area features in your models...). I recommend ‘GrLivArea’. Create a grid of ‘GrLivArea’ values between 0 and 15000. Next, loop through each value of ‘GrLivArea’ in the grid, and clone the training set at each step of the loop. In the cloned training set,

replace the real value of 'GrLivArea' with the grid value, and predict the log sales price on all the cloned data. Then compute the average of the predicted log sales price for each value of the 'GrLivArea'. Make a plot of the mean predicted log sales price versus 'GrLivArea' for both Random Forest and Ridge Regression. Explain what you observe. Note: Being aware of this phenomenon is crucial for properly using tree based methods.

Model Comparison

```
print(f"Ridge Test MSE: {mse:.4f}")
print(f"Random Forest Test MSE: {rf_mse:.4f}")
```

Ridge Test MSE: 0.0184

Random Forest Test MSE: 0.0162

Partial Dependence Plot for Gr Liv Area

```
# grid
gr_grid = np.linspace(0, 15000, 100)

feature_name = "Gr Liv Area"

ridge_curve = []
rf_curve = []

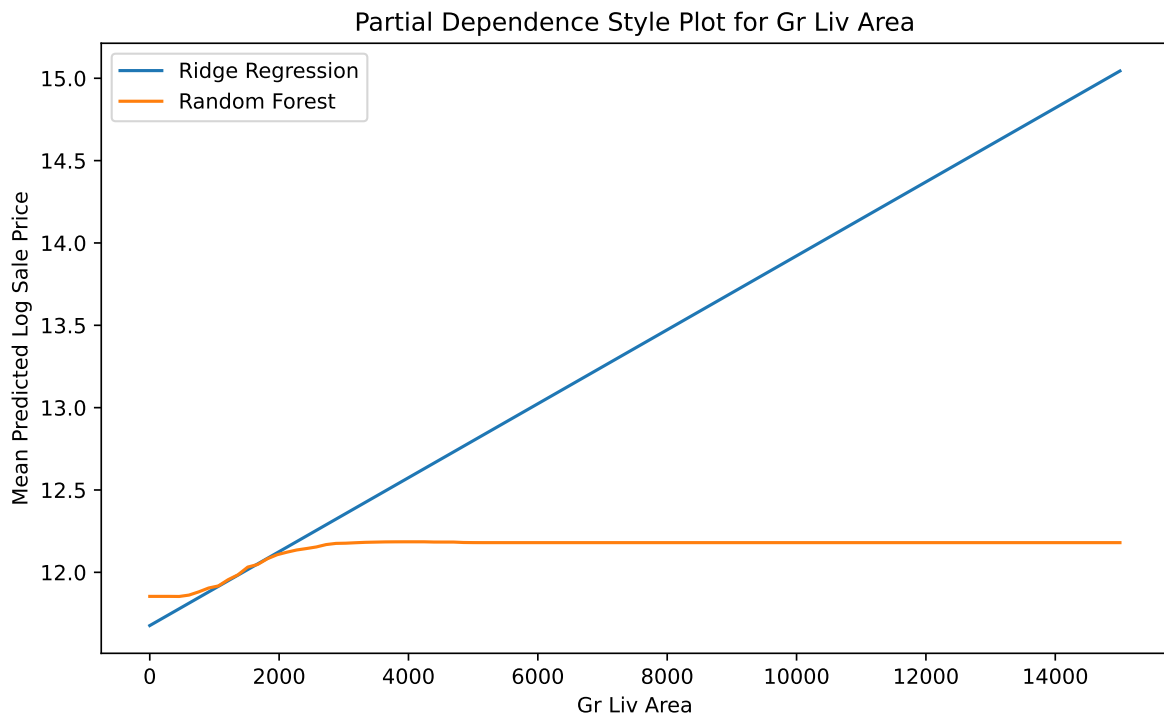
for val in gr_grid:
    X_clone = X_train.copy()
    X_clone[feature_name] = val

    ridge_preds = pipe_ridge.predict(X_clone)
    rf_preds = best_rf.predict(X_clone)

    ridge_curve.append(ridge_preds.mean())
    rf_curve.append(rf_preds.mean())

# plot
plt.figure(figsize=(8, 5))
plt.plot(gr_grid, ridge_curve, label="Ridge Regression")
plt.plot(gr_grid, rf_curve, label="Random Forest")
plt.xlabel("Gr Liv Area")
plt.ylabel("Mean Predicted Log Sale Price")
plt.title("Partial Dependence Style Plot for Gr Liv Area")
plt.legend()
```

```
plt.tight_layout()
plt.show()
```



The Random Forest probably captured some nonlinear structure since it achieved a slightly lower test error (0.0160) than Ridge regression (0.0184). In the partial dependence plot for Gr Liv Area, the Ridge model shows a smooth, steadily increasing relationship between living area and predicted sale price, reflecting its linear structure. In contrast, the Random Forest shows a much flatter, step-like pattern, where predictions increase early and then level off for larger values of living area. So, it keeps predicting the same value indefinitely. It can't extrapolate beyond the training data. So, tree-based models make piecewise predictions based on split regions, it won't extrapolate, while the ridge model assumes a the relationship will continue linear.

Problem 3: Boosting: Learning Rate and Tree Number

- (a) I strongly recommend using 'xgb' for this exercise. If you have a strong preference to stay within 'sklearn', you may use 'HistGradientBoostingRegressor', but it will be more effort to do part (b) properly. Fit a boosted model with the default hyperparameters. What is your average mean squared error and how does it compare to your other models?

Fit baseline boosted model

```
xgb_default = xgb.XGBRegressor(  
    random_state=42  
)  
  
xgb_default.fit(X_train, y_train)  
  
# Predictions  
y_pred_xgb = xgb_default.predict(X_test)  
  
# Test MSE  
xgb_mse = mean_squared_error(y_test, y_pred_xgb)  
  
print(f"XGBoost test MSE (log scale): {xgb_mse:.4f}")
```

XGBoost test MSE (log scale): 0.0174

Model Comparison

```
print("\n===== Test MSE Comparison =====")  
print(f"Ridge Test MSE: {mse:.4f}")  
print(f"Random Forest Test MSE: {rf_mse:.4f}")  
print(f"XGBoost: {xgb_mse:.4f}")
```

```
===== Test MSE Comparison =====  
Ridge Test MSE: 0.0184  
Random Forest Test MSE: 0.0162  
XGBoost: 0.0174
```

Using default hyperparameters, the XGBoost model achieved a test MSE of 0.0174, which performed better than Ridge regression (0.0184) but slightly worse than the Random Forest (0.0160). So, boosting improved on the linear model by capturing nonlinear relationships, but with default settings it did not outperform the tuned Random Forest. Boosting can be more sensitive to hyperparameters and may require tuning of the learning rate and number of trees to reach its best performance.

- (b) One of the differences between gradient boosted trees and bagging methods like random forest, is that boosted trees can sometimes be more powerful, but are also more sensitive

to hyperparameters and prone to overfitting. A fundamental trade-off in hyperparameters is between learning rate ('learning_rate') and the number of trees ('n_estimators'). For three values of the learning rate (pick the default of 0.3, one larger value, and one smaller value), plot the relationship between both training and testing mean-squared error and the number of boosting iterations. Make sure to pass the training and testing dataset as 'eval_set' when you are fitting your 'xgb' model, i.e. 'eval_set=[(X_train, y_train), (X_test, y_test)]'. The evolution of the error is tracked in 'results['validation_n']' where n corresponds to each validation set. For each learning rate, what is the best value of 'n_estimators' and the corresponding test mean squared error? *Note: passing the test set as eval_set is used here for visualization only. See the extra credit to learn how to do this properly with cross-validation.*

Learning Rate vs Number of trees

I am using 0.01 as the smaller learning rate value, 0.3 the default required, and 0.5 as the larger value.

```
learning_rates = [0.01, 0.30, 0.50]

results_dict = {}

for lr in learning_rates:

    model = xgb.XGBRegressor(
        learning_rate=lr,
        n_estimators=500,
        random_state=42
    )

    model.fit(
        X_train,
        y_train,

        eval_set=[
            (X_train, y_train),
            (X_test, y_test)
        ],

        verbose=False
    )

    results_dict[lr] = model.evals_result()
```

Plot Training and Test MSE

```

for lr in learning_rates:

    results = results_dict[lr]

    train_rmse = results["validation_0"]["rmse"]
    test_rmse = results["validation_1"]["rmse"]

    # square rmse -> mse
    train_mse = [x**2 for x in train_rmse]
    test_mse = [x**2 for x in test_rmse]

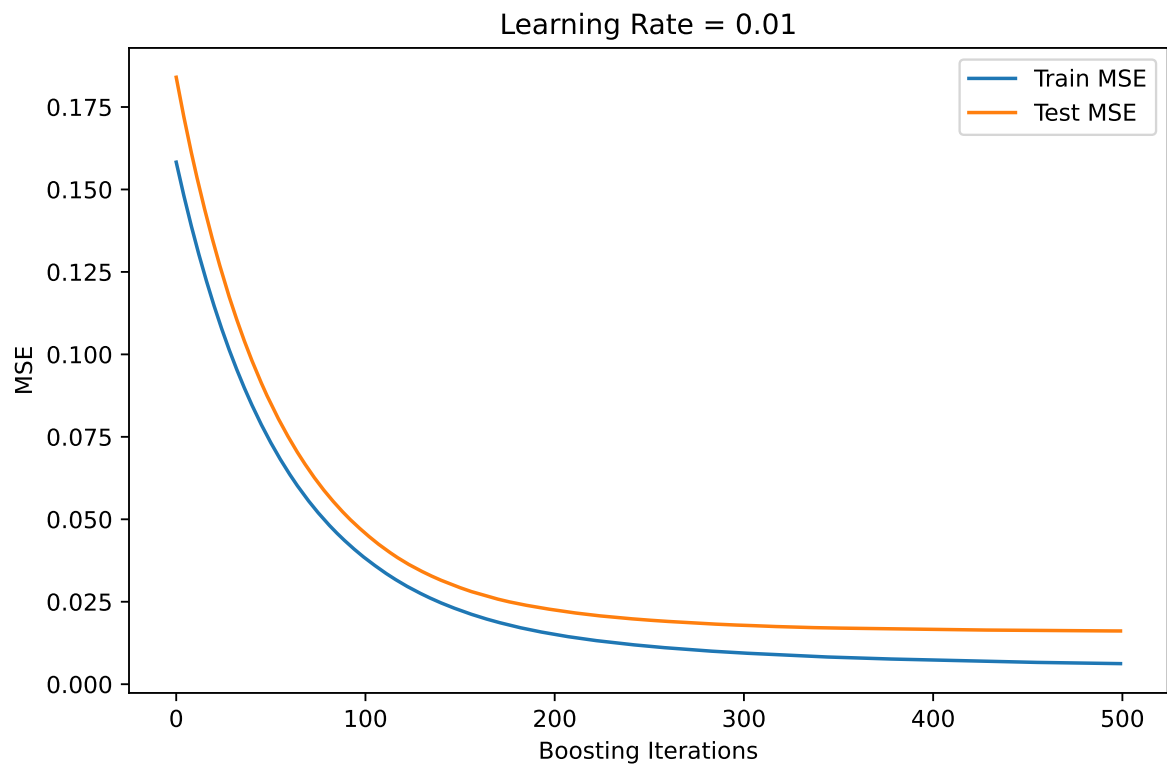
    plt.figure(figsize=(8,5))

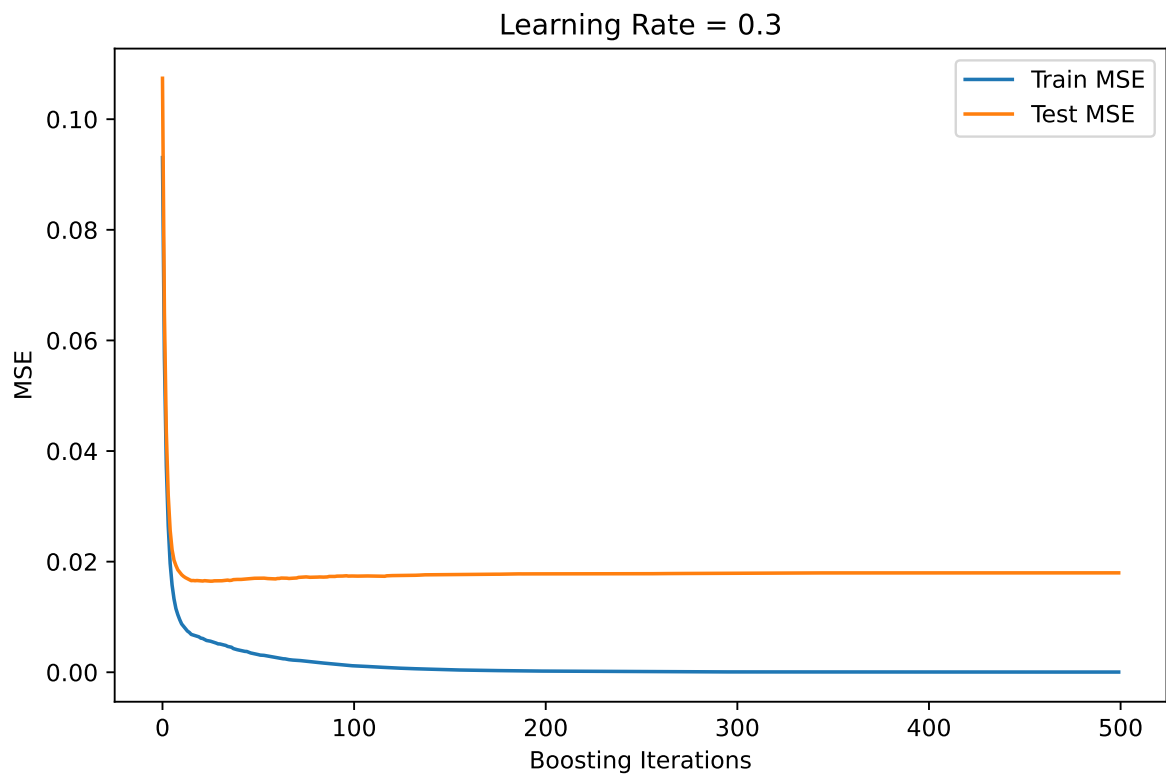
    plt.plot(
        train_mse,
        label="Train MSE"
    )

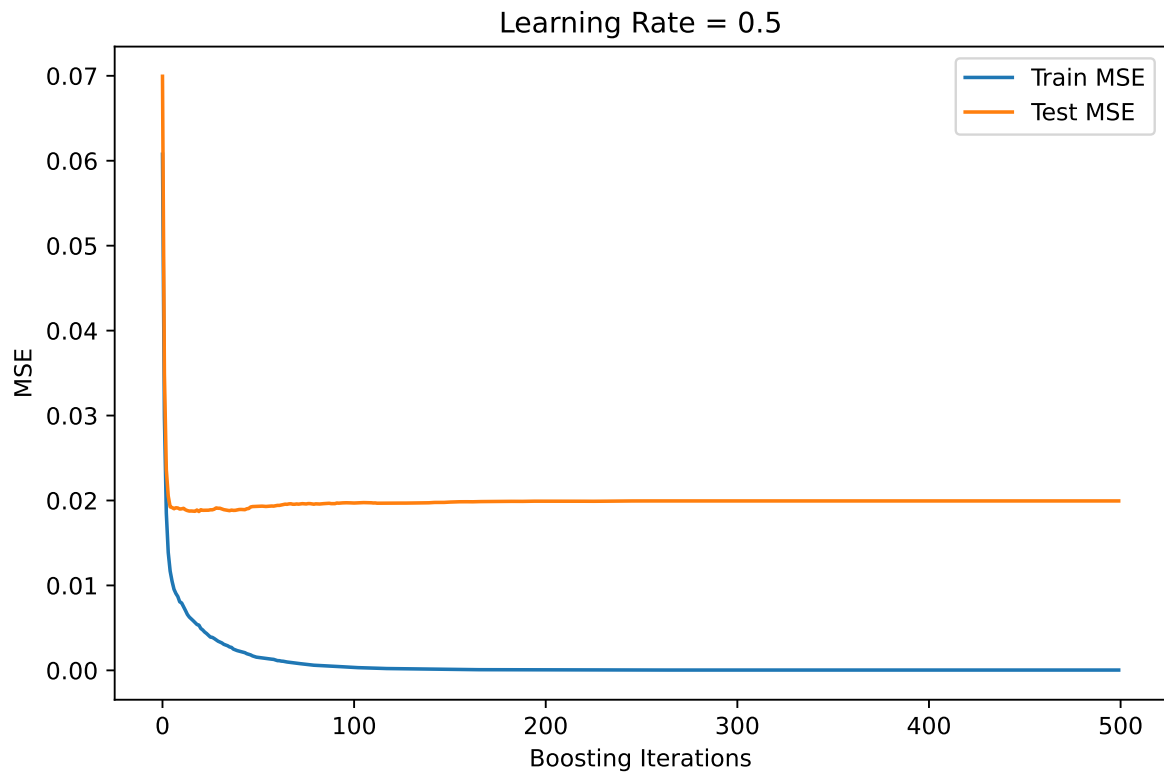
    plt.plot(
        test_mse,
        label="Test MSE"
    )

    plt.title(f"Learning Rate = {lr}")
    plt.xlabel("Boosting Iterations")
    plt.ylabel("MSE")
    plt.legend()
    plt.show()

```







Best Number of Trees for each Learning Rate

```
for lr in learning_rates:

    results = results_dict[lr]

    test_mse = [
        x**2 for x in results["validation_1"]["rmse"]
    ]

    best_iter = np.argmin(test_mse)

    print(
        f"Learning rate {lr}: "
        f"Best n_estimators = {best_iter+1}, "
        f"Best Test MSE = {test_mse[best_iter]:.4f}"
    )
```

Learning rate 0.01: Best n_estimators = 500, Best Test MSE = 0.0161

Learning rate 0.3: Best n_estimators = 26, Best Test MSE = 0.0165

Learning rate 0.5: Best `n_estimators` = 20, Best Test MSE = 0.0187

As expected there is a tradeoff between learning rate and number of boosting iterations. With the smaller learning rate of 0.01, the model improved gradually and continued getting better with more trees, achieving the best performance at 500 estimators with a test MSE of 0.0161, which was the best result. With the default learning rate (0.3), the model reached its best performance much earlier at 26 estimators with a test MSE of 0.0165, then began to level off. With the larger learning rate (0.5), the model reached its best result very quickly at 20 estimators, but had the worst performance (0.0187), suggesting it overfit easier. We can see in the training and testing error curves that smaller learning rates require more trees but can generalize better, while larger learning rates learn faster but are more prone to overfitting. So, the conclusion, in this case, is that a lower learning rate paired with more trees produced the best predictive performance.

Extra Credit (5 pts): Early Stopping

- (a) The relationship you found in 3(b) suggests a technique called *early stopping*, where you stop training the ‘xgboost’ model if there hasn’t been an improvement in the performance on the validation set for some number of iterations. This suggests using hyperparameter optimization to select the value of the learning rate, while implementing early stopping. Implementing this properly within a cross-validation loop can be slightly tricky, because you need to pass the held out data in the cross-validation loop to the fitting routine as the ‘eval_set’, which is not automatically handled by the standard ‘sklearn’ methods for hyperparameter optimization. The following article discusses this issue and shows how to implement this by hand (option 2 in his code): [Jeff Macaluso on Early Stopping](#). Implement a similar approach to pick the best value of the learning rate. I recommend tuning just the learning rate using a grid search approach, but if you want to try more hyperparameters and a randomized search, you are welcome to do so but it will be computationally more intensive (this is what Macaluso does). We will learn more methods for hyperparameter optimization later in the course. Report how the test MSE and the number of trees selected by early stopping varied across learning rates, and compare the test mse to your other three models (the default xgboost and the ridge regression and random forest). Make sure to follow Macaluso’s prescription for training your final model, by holding out 15% of the training set to recalculate the number of trees, and then evaluating that final model on the test set.

Early Stopping with manual CV

```
learning_rates = [0.01, 0.05, 0.10, 0.30, 0.50]
kf = KFold(n_splits=5, shuffle=True, random_state=42)

cv_results = []
```

```

for lr in learning_rates:
    fold_mse = []
    fold_best_iters = []

    for train_idx, val_idx in kf.split(X_train):
        X_tr = X_train.iloc[train_idx]
        y_tr = y_train.iloc[train_idx]
        X_val = X_train.iloc[val_idx]
        y_val = y_train.iloc[val_idx]

        model = xgb.XGBRegressor(
            learning_rate=lr,
            n_estimators=5000,
            early_stopping_rounds=50,
            random_state=42,
            objective="reg:squarederror",
            eval_metric="rmse"
        )

        model.fit(
            X_tr,
            y_tr,
            eval_set=[(X_val, y_val)],
            verbose=False
        )

        best_iter = model.best_iteration + 1
        val_pred = model.predict(X_val)
        val_mse = mean_squared_error(y_val, val_pred)

        fold_mse.append(val_mse)
        fold_best_iters.append(best_iter)

    cv_results.append({
        "learning_rate": lr,
        "cv_mse_mean": np.mean(fold_mse),
        "cv_mse_std": np.std(fold_mse),
        "avg_best_n_estimators": int(round(np.mean(fold_best_iters))),
        "median_best_n_estimators": int(np.median(fold_best_iters))
    })

cv_results_df = pd.DataFrame(cv_results).sort_values("cv_mse_mean")

```

```
print(cv_results_df)
```

	learning_rate	cv_mse_mean	cv_mse_std	avg_best_n_estimators \
1	0.05	0.022450	0.007975	274
2	0.10	0.022539	0.008330	148
0	0.01	0.022697	0.008146	940
3	0.30	0.024300	0.008873	38
4	0.50	0.025990	0.007735	26

	median_best_n_estimators
1	300
2	151
0	949
3	43
4	24

Select the best Learning Rate

```
best_lr = cv_results_df.iloc[0]["learning_rate"]
print("Best learning rate from CV:", best_lr)
```

Best learning rate from CV: 0.05

```
X_tr_final, X_val_final, y_tr_final, y_val_final = train_test_split(
    X_train, y_train,
    test_size=0.15,
    random_state=42
)

final_es_model = xgb.XGBRegressor(
    learning_rate=best_lr,
    n_estimators=5000,
    early_stopping_rounds=50,
    random_state=42,
    objective="reg:squarederror",
    eval_metric="rmse"
)

final_es_model.fit(
    X_tr_final,
```

```

    y_tr_final,
    eval_set=[(X_val_final, y_val_final)],
    verbose=False
)

final_best_n = final_es_model.best_iteration + 1
print("Final early-stopped number of trees:", final_best_n)

```

Final early-stopped number of trees: 161

Refit on the Full Training Set

```

final_model = xgb.XGBRegressor(
    learning_rate=best_lr,
    n_estimators=final_best_n,
    random_state=42,
    objective="reg:squarederror",
    eval_metric="rmse"
)

final_model.fit(X_train, y_train, verbose=False)

y_pred_final = final_model.predict(X_test)
final_test_mse = mean_squared_error(y_test, y_pred_final)

print("Final early-stopped XGBoost test MSE:", final_test_mse)

```

Final early-stopped XGBoost test MSE: 0.01594511291232026

Final Models Comparison

```

comparison_df = pd.DataFrame({
    "Model": [
        "Ridge Regression",
        "Random Forest",
        "Default XGBoost",
        "Early-Stopped XGBoost"
    ],
    "Test_MSE": [
        0.0184,

```

```

        0.0160,
        0.0174,
        final_test_mse
    ]
})

print(comparison_df.sort_values("Test_MSE"))

```

	Model	Test_MSE
3	Early-Stopped XGBoost	0.015945
1	Random Forest	0.016000
2	Default XGBoost	0.017400
0	Ridge Regression	0.018400

```

report_df = cv_results_df.copy()
report_df["final_test_mse"] = np.nan
report_df.loc[report_df["learning_rate"] == best_lr, "final_test_mse"] = final_test_mse
print(report_df)

```

	learning_rate	cv_mse_mean	cv_mse_std	avg_best_n_estimators	\
1	0.05	0.022450	0.007975	274	
2	0.10	0.022539	0.008330	148	
0	0.01	0.022697	0.008146	940	
3	0.30	0.024300	0.008873	38	
4	0.50	0.025990	0.007735	26	

	median_best_n_estimators	final_test_mse
1	300	0.015945
2	151	NaN
0	949	NaN
3	43	NaN
4	24	NaN

I implemented early stopping manually in a 5 fold CV loop. The results showed a pattern where smaller learning rates required more trees, while larger learning rates stopped much earlier. For example, a learning rate of 0.01 used about 940 trees on average, while 0.50 used only about 26. In terms of cross-validated performance, the best learning rate was 0.05, with an average CV MSE of 0.02245, slightly better than 0.10 and 0.01, and clearly better than the larger learning rates. Then, I held out 15% of the training set to estimate again the best number of trees for the chosen learning rate. With learning rate = 0.05, early stopping selected

161 trees for the final model. When this final early-stopped XGBoost model was evaluated on the test set, it achieved a test MSE of 0.01595, which was the best result among all models I fit. The early-stopped XGBoost slightly outperformed the Random Forest (0.0160), and did better than both the default XGBoost (0.0174) and Ridge regression (0.0184). The results show that tuning the learning rate together with early stopping improved boosting performance and helped control overfitting.